STANDARD OPERATING PROCEDURE

Platform Delivery SOP
User App • Vendor App • Admin Panel • Website

With Live Tracking — 8 to 10 Day Delivery Plan, based on analysis of the existing codebase

Document	SOP-PLT-001 | Version 1.0 (Draft for client review)
Prepared by	Aryan Pathak (Delivery / Project Lead)
Prepared for	Management & Client Stakeholders
Date	12 October 2026
Basis	Review of repository `E:\CarCommunity` — `backend-labourchouck` (Node.js API) and `frontend` (React web app)
Target timeline	8–10 working days from kick-off

1. Document Control & Contents

1.1 Purpose of this SOP

This SOP defines **how** the team will take the existing codebase and deliver the four client-required products — **User Application, Vendor Application, Admin Panel and Website**, all with **live tracking** — in 8–10 days. It covers what already exists, what must be adapted, what must be built, who does what, in which order, and how each stage is verified and signed off.

1.2 Who should use it

Reader	How they use this document
Client / Business owner	Sections 2, 4, 6, 9, 14 — understand scope, timeline, acceptance criteria.
Project lead	Entire document — plan, track, report, control change.
Frontend / Backend developers	Sections 3, 4, 5, 8, 9 — daily task list and technical approach.
QA / Tester	Sections 6, 11 — test matrix and UAT checklist.
DevOps	Section 12 — environment, deployment, go-live and rollback.

1.3 Contents

No.	Section
1	Document control & contents
2	Executive summary & project understanding
3	Analysis of the existing system (architecture, roles, modules, data model)
4	Scope of Work — the four deliverables
5	Live tracking — current state, gaps, and design
6	Feature matrix — Ready / Adapt / Build
7	Role-wise workflows (User, Vendor, Admin)
8	UI/UX adaptation SOP
9	10-day execution plan (day-by-day)
10	Team, responsibilities & communication rhythm
11	Quality assurance & UAT SOP
12	Deployment, go-live & rollback SOP
13	Risks, assumptions, dependencies & out-of-scope
14	Change control, acceptance criteria & sign-off

1.4 Revision history

Version	Date	Author	Change
1.0	12 Oct 2026	Aryan Pathak	Initial draft from codebase analysis

2. Executive Summary & Project Understanding

2.1 What the client has asked for

From the client conversation, the requirement is a complete on-demand service platform consisting of:

1. **User Application** — for end customers to browse services, book, pay and track.
2. **Vendor Application** — for service providers/contractors to receive, accept, assign and complete jobs and get paid.
3. **Admin Panel** — for the business to control users, vendors, pricing, bookings, money and content.
4. **Website** — public marketing/landing site that drives customers and vendors into the apps.
5. **Live tracking** across the booking journey, plus the "features as discussed".
6. A UI/UX matching the reference design shared by the client, delivered in **8–10 days**.

2.2 Key finding from the analysis

> **We are not starting from zero.** The repository already contains a working, feature-rich multi-role platform: a Node.js/Express + MongoDB backend with about 40 route modules and 45 data models, and a React 19 web application containing the public website, a mobile-style user/worker app shell, a vendor (contractor) panel, a corporate panel and a 40-page admin panel. Real-time (Socket.io), payments (Razorpay), push notifications (Firebase FCM), wallet, commission, referral, subscriptions, OTP-verified job start/end and a materials marketplace ("BuildMart") are already implemented.

Therefore the 8–10 day plan is realistic **only** because the strategy is **adapt & harden, not rebuild**: re-skin the UI to the client's reference design, align terminology and flows to the client's business, complete and harden live tracking, close the gaps listed in Section 6, then test and deploy.

2.3 Delivery strategy in one view

Principle	What it means in practice
Reuse first	Every module that already works is kept; changes are configuration, copy and styling wherever possible.
Vertical slices	Work is organised by journey (Book → Track → Pay) not by file, so each day ends with something demonstrable.
Live tracking is the hero	It gets dedicated engineering time (Section 5) because it is the one feature the client named explicitly.
Freeze early	Scope is frozen on Day 1 sign-off; anything new goes through change control (Section 14).
Test continuously	QA starts on Day 3, not Day 8; UAT on Day 8 is confirmation, not discovery.

2.4 Success criteria

- A user can register, book, pay, watch the provider approach on a live map, complete the job and rate it — end to end, without manual intervention.
- A vendor can be onboarded (KYC), approved, receive jobs, accept/assign, work, and withdraw earnings.
- Admin can configure pricing/commission, approve vendors, monitor all bookings live and settle money.
- The website is live, responsive and routes visitors to the correct app.
- All Priority-1 test cases pass on the production environment before hand-over.

3. Analysis of the Existing System

3.1 Technology stack

Layer	Technology (from `package.json`)	Notes
Backend	Node.js (ES modules), Express 4, Mongoose 8 (MongoDB), express-validator, Helmet, Morgan, Multer	REST API, versioned under one router (`routes/index.js`)
Real-time	Socket.io 4 with JWT-authenticated sockets	Each user joins a private room by user ID
Auth	JWT, bcryptjs, OTP login (SMS gateway, DLT template)	Dev OTP bypass flags exist — must be off in production
Payments	Razorpay (orders, verify, wallet clearance)	Online + Cash flows both supported
Notifications	Firebase Admin (FCM push), in-app notifications	Needs live service-account key
Media	Cloudinary uploads	KYC docs, job photos, banners
Scheduling	node-cron (every-minute broadcast & stuck-booking cleanup)	Scheduled bookings start broadcasting 1 hour before time
Frontend	React 19, Vite 8, Redux Toolkit, React Router 7, Tailwind CSS 4, Framer Motion, Lottie, socket.io-client, Firebase	Single SPA, role-based routing, deployable on Vercel (`vercel.json`)
Maps	Google Maps Places Autocomplete + browser Geolocation	Requires `GOOGLE_MAPS_API_KEY`

3.2 User roles in the system

Role (code)	Maps to client term	What they do today
`individual`	**User** (customer)	Browse catalog, instant/scheduled booking, wallet, referral, subscription, marketplace enquiry.
`labour`	Service worker	Receives broadcast jobs, accepts, OTP start/finish, earns, withdraws, KYC.
`contractor`	**Vendor**	Business profile + KYC, manages crew, accepts/assigns jobs, attendance, earnings, subscriptions, marketplace products.
`corporate`	B2B client (optional)	Projects/sites, bulk workforce requests, invoices, analytics.

`admin`	**Admin**	Full control: users, vendors, catalog, pricing, wallets, bookings, reports, CMS.

> **Decision needed (Day 1):** the client names only *User, Vendor, Admin*. The existing `labour` (individual worker) and `corporate` roles are extra. We recommend keeping them hidden/behind feature flags unless the client confirms they are needed — this protects the 8–10 day timeline.

3.3 Backend module inventory

Identity & onboarding
Auth/OTP, users, vendor KYC & documents, corporate verification, address book, crew labour management with OTP linking.

Catalog & pricing
Labour category groups → categories → subcategories → services, pricing rates, time slots, zones, banners.

Booking engine
Calculate bill, create booking (instant/scheduled), broadcast to nearby providers, accept/reject, status machine, daily start/end OTP, cancellation.

Money
Wallets (earnings + dues), commission, platform fee, wallet discount, withdrawals, invoices, settlements, referral rewards, subscriptions.

Workforce / B2B
Workforce requests, allocations, assignments, attendance (4-stage check-in/out), projects & sites, billing.

Engagement & support
Reviews, complaints, notifications (push + in-app), FAQs, terms, privacy policy, referral.

Marketplace (BuildMart)
Categories, products, banners, product reviews, lead/enquiry capture, vendor product CRUD, mart subscriptions.

Admin & reporting
Dashboard stats, reports, settings (fees, radius, limits), CMS content, wallet & withdrawal approvals.

3.4 Core booking lifecycle (as implemented)

CREATED → **BROADCASTING** (nearby eligible providers receive `NEW_BROADCAST`) → **ACCEPTED** (first accept wins) → **ASSIGNED** → **EN_ROUTE** → **STARTED** (start OTP verified) → **COMPLETED** (end OTP verified) | exits: **CANCELLED**, **FAILED** (no provider within 10 minutes — auto, with wallet refund), **REFUNDED**.

Money is computed at booking time and stored on the booking: `basePrice`, `platformFee`, `taxes`, `totalAmount`, `commissionAmount`, `laborShare`, optional `walletDiscount`. Admin-configurable settings include platform fee (fixed/percent), B2B fee, global commission %, broadcast radius (10 km B2C, 50 km B2B), wallet/cash limits, cancellation penalty and time slots.

3.5 Frontend structure

- **Website:** `LandingPage`, legal pages, boot/splash and role-select screens.
- **User/worker app shell (`/app`):** ~30 pages — home, services, sub-category, checkout, tracking, bookings, wallet, refer & earn, KYC, earnings, notifications, BuildMart, address book, support.
- **Vendor panel (`/vendor`):** dashboard, jobs, job detail, crew, attendance, earnings, analytics, mart products/enquiries/subscription, profile, support.
- **Admin panel:** ~40 pages — dashboard, users, vendors, labour, bookings, categories/subcategories/services, zones, time slots, fees, commission, wallets & cash, banners, reviews, complaints, referrals, subscriptions, reports, CMS, settings.
- **Documentation present:** `frontend/docx` (API contract, Postman collection, developer guide) and backend API notes (Vendor, Admin Booking, Job Allocations, BuildMart).

3.6 Housekeeping observations

- The backend root holds many one-off scripts (`check_*.js`, `fix_*.js`, `test*.js`) — they are data-repair utilities and must not ship to production.
- `.env` and a Firebase service-account JSON are present in the project — secrets must be rotated and kept out of version control and hand-over packages.
- Several comments in routes/models show the product was built for a labour-hiring business; user-facing wording, seed data and icons must be re-aligned to the client's business (Section 8).

4. Scope of Work — The Four Deliverables

Each deliverable below lists the **baseline we inherit**, the **work to be done** in this engagement, and the **exit condition**.

4.1 User Application

Area	Baseline (exists)	Work in this engagement
Onboarding & login	Splash, role select, mobile-OTP auth, guest browsing	Apply new branding; simplify to User-only entry; production SMS OTP (bypass off).
Discovery	Home with banners, category → subcategory → service pages, search	Re-skin to reference UI; load client's catalog and imagery.
Booking	Instant & scheduled flow, address book, Places autocomplete, bill calculation, time slots	Align flow/steps to reference UI; validate pricing rules per client.
Payment	Razorpay online, cash, wallet discount	Switch to client's live Razorpay keys; test refunds.
Live tracking	Partial — location event stream + tracking page	Complete map view, ETA, status stepper, reconnect handling (Section 5).
Post-service	OTP completion, reviews, complaints	Rating modal polish; invoice/receipt view.
Account	Wallet, referral, subscription, notifications, FAQ/T&C	Content update; hide unused items.

Exit condition: a new user completes a paid booking and sees the provider live on a map on a real phone.

4.2 Vendor Application

Area	Baseline (exists)	Work in this engagement
Registration & KYC	Business profile, PAN/GST/documents, verification checklist & submit	Align document list to client; admin approval loop test.
Job intake	Real-time job popup, accept/reject, direct requests, availability toggle	Sound/vibration alert, timer UX; mobile-first layout.
Execution	Assign/replace crew, OTP start & end, photos, attendance, cash collection	Add **vendor live-location sharing** from the job screen (Section 5).
Earnings	Dashboard, earnings, analytics, settlements, withdrawals with bank details	Verify payout math; admin approval loop.
Growth	Subscription plans, BuildMart product listing & enquiries	Keep only if client confirms; otherwise feature-flag off.

Exit condition: a vendor is onboarded, approved, accepts a job, shares location, completes with OTP and requests a withdrawal that admin approves.

> **Note on "Vendor Application":** today the vendor experience is a web panel (`/vendor`) inside the same React app, designed mobile-first. Delivering it as an installable **PWA** satisfies "application" within 10 days. A native Play Store / App Store build is **not** in this timeline unless explicitly added (see Section 13).

4.3 Admin Panel

Area	Baseline (exists)	Work in this engagement
Dashboard & reports	Stats, reports, revenue views	Add **live operations view** (active jobs on map).
User & vendor management	List, view, block, delete, vendor approval, KYC review	Wording and field alignment.
Catalog & pricing	Categories, subcategories, services, zones, time slots	Load client catalog; confirm price ladder.
Money control	Platform fee, commission, wallet limits, cash management, withdrawals, referrals, subscriptions	Configure client values; reconcile one sample booking end-to-end.
Content / CMS	Banners (user/vendor/corporate/mart), FAQ, T&C, privacy	Replace default legal text with client-approved text.
Support	Complaints, reviews & ratings	Verify status workflow.

Exit condition: admin can run the business for a day without touching the database.

4.4 Website

Area	Baseline (exists)	Work in this engagement
Landing page	Animated landing page, role entry points	Rebuild sections/copy/imagery per reference UI.

Conversion	Links into app/vendor flows	Clear "Book now", "Become a vendor", app-download CTAs.
Trust & legal	Terms, privacy pages, FAQ	Final legal text, contact details, social links.
SEO & speed	react-helmet-async present	Titles/meta, OG tags, image optimisation, sitemap.

Exit condition: website live on the client domain with HTTPS, responsive on mobile/tablet/desktop, Lighthouse performance ≥ 80.

5. Live Tracking — Current State, Gaps and Design

5.1 What already exists (verified in code)

Piece	Where
Worker device sends GPS	`ActiveJob.jsx` uses the browser `watchPosition` API while a job is active.
Socket relay to customer	`socket.js` listens for `LABOUR_LOCATION_UPDATE` and forwards it to the customer's private room.
Customer receives updates	`JobTracking.jsx` subscribes to `LABOUR_LOCATION_UPDATE`, `BOOKING_ACCEPTED`, `BOOKING_STATUS_UPDATE`.
Last-known location storage	REST `POST /labour/location/update` and `/status` (provider location & online status; used for nearby matching).
Geospatial matching	Bookings carry GeoJSON coordinates with a 2dsphere index; broadcast radius is admin-configurable.
Push fallback	FCM notifications for accept / status / failure.

5.2 Gaps to close

#	Gap	Why it matters / fix
1	Location relay trusts `customerId` sent by the client	**Security:** any logged-in socket could push fake locations to any user. Fix: server looks up the booking, verifies the sender is the assigned provider, and derives the customer ID itself.
2	No emission when the vendor (contractor) is the assigned party	Add the same location sharing to the vendor job screen and to the assigned crew member where relevant.
3	No stored route / breadcrumb	Persist throttled points per booking so admin can replay and resolve disputes.
4	No ETA / distance to destination	Compute from Google Directions/Distance Matrix (server-side, cached) and show "Arriving in X min".
5	Map visual on tracking page to be confirmed/completed	Show provider marker, destination marker, route line, smooth marker animation.
6	Reconnect & refresh behaviour	On reload, fetch last known point via REST, then resume socket stream. Persist active booking ID.
7	Update frequency / battery	Throttle to one update every 5–10 s or 20 m movement; stop streaming when status is not EN_ROUTE/STARTED.
8	Admin live view absent	Admin map of all active bookings with provider position and status.
9	Background tracking limits of browser/PWA	Web geolocation pauses when the screen locks. Document clearly; mitigate with wake-lock and keep-screen-on prompt on the job screen.

5.3 Target design

Provider device — GPS every 5–10 s → `socket: LOCATION_UPDATE {bookingId, lat, lng, heading, speed}` → **Server** validates sender == assigned provider and booking status ∈ {EN_ROUTE, STARTED} → saves latest point + throttled breadcrumb → computes ETA (cached) → emits to **customer room** and **admin live room** → **Customer map** animates marker, updates ETA, shows status stepper.

Acceptance tests for live tracking

- Provider moves 500 m → customer marker moves within 3 seconds.
- Customer refreshes the page → marker reappears at last known position within 2 seconds.
- A third user cannot receive, and cannot inject, another booking's location.
- Streaming stops automatically at COMPLETED / CANCELLED.

- ETA shown is within ±20% of Google Maps for the same route.

6. Feature Matrix — Ready / Adapt / Build

Legend: READY works as-is, needs config/testing only ADAPT exists, needs UI/flow/wording change BUILD new or substantial completion. Priority P1 = must for go-live, P2 = should, P3 = nice to have.

Feature	Product	Status	Prio	Remark
Mobile OTP login	User, Vendor	READY	P1	Turn OFF OTP bypass flags; configure DLT SMS.
Service catalog & search	User	ADAPT	P1	Load client categories, icons, prices.
Instant booking + broadcast	User, Vendor	READY	P1	Tune radius, timer, retry rules.
Scheduled booking	User	READY	P1	Cron triggers 1 h before slot.
Bill calculation (fee, tax, commission)	All	ADAPT	P1	Confirm GST, fee and commission rules with client.
Online payment (Razorpay)	User	READY	P1	Client live keys; webhook/verify test.
Cash payment & dues	Vendor, Admin	READY	P1	Cash limit blocks new jobs when exceeded.
Live tracking (map, ETA)	User, Vendor, Admin	BUILD	P1	Section 5.
OTP start / end of job	User, Vendor	READY	P1	Daily OTP for multi-day jobs.
Ratings & reviews	User	READY	P2	Admin moderation page exists.
Wallet, withdrawals, settlements	User, Vendor, Admin	READY	P1	Reconcile one booking end-to-end.
Vendor KYC & approval	Vendor, Admin	ADAPT	P1	Align document list.
Crew management & assignment	Vendor	READY	P2	Keep if vendors have staff.
Push notifications	All	ADAPT	P1	New Firebase project & keys; copy review.
Referral & subscriptions	User, Vendor	READY	P3	Feature-flag per client decision.
BuildMart marketplace	All	ADAPT	P3	Out of scope unless confirmed; hide.
Corporate / B2B panel	Corporate	ADAPT	P3	Out of scope unless confirmed; hide.
Admin live operations map	Admin	BUILD	P2	Reuses tracking stream.
Admin CMS & settings	Admin	READY	P1	Fill values.
Complaints & support	User, Admin	READY	P2	Verify workflow.
Website landing + SEO	Website	ADAPT	P1	Rebuild sections per design.
PWA install (vendor/user)	Apps	BUILD	P2	Manifest, icons, service worker.
Production hardening (rate limit, logs, secrets)	Backend	BUILD	P1	Section 12.

7. Role-wise Workflows

7.1 User journey

1. **Open website / app** → splash → choose to browse as guest or log in.
2. **Login** with mobile number → OTP → profile created.
3. **Choose service** → category → sub-category → service.
4. **Set address & time** → map pin / autocomplete → Instant or Scheduled slot.
5. **Review bill** → base price + platform fee + tax = total; optional wallet credit.
6. **Pay** → Online (Razorpay) or Cash.
7. **Searching** → broadcast to nearby providers; user sees “Finding your provider”.

8. **Accepted** → provider name, photo, phone, rating shown; **live map** starts when EN_ROUTE.
9. **Arrival & start** → user shares start OTP; status STARTED.
10. **Completion** → end OTP; payment settles; rating & review prompt; receipt.
11. **After** → booking history, rebook, complaint, referral.

User edge cases

- No provider accepts in 10 minutes → booking FAILED, wallet discount refunded, user notified.
- User cancels before accept → free; after accept → policy/penalty as configured.
- Payment failed → booking stays unpaid; retry or switch to cash.

7.2 Vendor journey

1. **Register** → business details → upload KYC documents → submit for verification.
2. **Wait for approval** → admin approves or rejects with reason; vendor notified.
3. **Go online** → toggle availability; location shared for matching.
4. **Receive job** → popup with service, distance, earning, countdown → Accept / Reject.
5. **Assign crew** (if applicable) → select member; replace if unavailable.
6. **Navigate** → Start journey (EN_ROUTE), live location shared, "Open in Google Maps".
7. **Start work** → enter customer start OTP + start photo.
8. **Finish work** → end OTP + end photo; collect cash if cash booking.
9. **Earnings** → wallet credit minus commission; withdraw to bank; admin approves.

Vendor edge cases

- Cash dues above limit → vendor blocked from new jobs until cleared online.
- Cancel after accept → penalty debited from wallet.
- Offline mid-job → last location retained; customer sees "location last updated X min ago".

7.3 Admin journey

1. **Login** → dashboard KPIs (bookings, revenue, active vendors, pending approvals).
2. **Approve vendors** → review documents → approve/reject.
3. **Configure** → categories, services, prices, zones, slots, fee, commission, radius, limits.
4. **Monitor** → all bookings, live map, intervene (reassign/cancel/refund) via admin booking API.
5. **Finance** → wallets, cash collected, withdrawals approve/reject, commission report.
6. **Content & support** → banners, FAQ, legal; complaints; reviews.
7. **Reports** → export for accounting.

7.4 End-to-end sequence

> User books → System broadcasts → Vendor accepts → User + Admin notified → Vendor EN_ROUTE (live map) → Start OTP → Work → End OTP → Payment settled (commission split) → Ratings → Wallet / withdrawal → Admin reports.

8. UI/UX Adaptation SOP

The client wants the UI/UX "like this" (reference design shared in the conversation). The existing app already has a design system (`components/app-ui`: buttons, cards, inputs, navigation, feedback, typography; Tailwind 4 theme tokens in `src/themes`). The goal is **re-skin through tokens and component variants**, not rewriting screens.

8.1 Procedure

Step	Action	Output
1	Collect reference design (all screens/links) and brand assets (logo, colours, fonts)	Design pack in shared folder
2	Screen mapping: reference screen → existing page → delta (None / Style / Layout / New)	Mapping sheet (Day 1)
3	Update design tokens: colours, radius, shadows, typography, spacing, icon set	Theme file merged
4	Update shared components first (button, card, input, header, bottom nav, sheet, toast)	Component gallery verified

5	Page-by-page adaptation in priority order: Home → Catalog → Checkout → Tracking → Bookings → Wallet → Profile	Pages signed off in review
6	Admin panel: apply colour/typography only; keep layout (tables/forms) as is	Consistent look, low risk
7	Responsive pass at 360, 390, 768, 1280 px	No horizontal scroll, tap targets ≥ 44 px
8	Content pass: replace labour-specific wording, icons, seed data, legal text	Copy deck approved by client

8.2 Design rules

- One primary action per screen; sticky bottom CTA on mobile.
- Booking status always visible as a stepper; colours: pending (amber), active (blue), done (green), failed/cancelled (red).
- Skeleton loaders for lists, empty-state illustration with action for every list.
- Accessible contrast (WCAG AA), minimum 14 px body text on mobile.
- Maps: full-bleed with bottom sheet containing provider card, ETA, call/chat, OTP.
- Motion: subtle (Framer Motion), never block the user; respect reduced-motion.

8.3 Screen priority (adapt in this order)

Prio	Screens	Reason
1	Splash, login/OTP, home, service listing	First impression; client will judge by these.
2	Checkout, payment, searching, tracking, OTP, completion/rating	Core revenue and live-tracking journey.
3	Vendor dashboard, job popup, job detail, earnings, KYC	Supply side must be equally polished.
4	Website landing, legal pages	Marketing entry.
5	Admin panel theme	Internal; lowest visual risk.

9. 10-Day Execution Plan

Day 0 = kick-off. The plan has two spare days (Day 9–10) as buffer; if everything is on time the project can close on Day 8.

Day	Theme	Tasks	Deliverable / Gate
Day 1	Freeze & setup	Scope workshop; confirm roles to keep (Section 3.2); collect reference UI, brand assets, API keys (Razorpay, Google Maps, Firebase, SMS); set up staging environment; screen-mapping sheet; feature flags for out-of-scope modules.	**Gate 1:** signed scope & asset pack
Day 2	Foundation	Design tokens & shared components; clean repo (remove scripts/secrets); env configs; seed client catalog, zones, time slots, admin account; production OTP wiring.	Staging running with client data
Day 3	User app UI	Splash, login, home, catalog, sub-category pages re-skinned; address & map picker; copy changes.	User app demo build (part 1)
Day 4	Booking & payment	Checkout, bill, slots, payment flow, bookings list; Razorpay live-mode test; wallet discount check.	Paid booking works on staging
Day 5	Live tracking (backend)	Secure location relay, booking-bound validation, breadcrumb storage, ETA service, admin room, reconnect REST endpoint, throttling.	Tracking API contract + tests
Day 6	Live tracking (UI) + Vendor	Customer map screen, provider marker, ETA, stepper; vendor job screen sends location; vendor KYC, job popup, earnings UI re-skin.	**Gate 2:** live tracking demo end-to-end
Day 7	Admin & website	Admin theme + live operations map; configure fees/commission; website landing, SEO, legal pages; PWA manifest/icons.	All 4 products feature-complete
Day 8	QA & UAT	Full regression, device matrix, payment & refund cases, load smoke test; client UAT walkthrough; bug triage.	**Gate 3:** UAT report
Day 9	Fix & harden	Fix P1/P2 bugs; security checks (rate limits, CORS, secrets rotated, helmet); backups; monitoring; content final.	Release candidate
Day 10	Go-live	Production deploy, DNS/SSL, smoke test on production, hand-over docs, training session, hyper-care plan.	**Gate 4:** go-live sign-off

9.1 Daily operating rhythm

- **10:00** — 15-minute stand-up: yesterday / today / blockers.
- **Throughout** — small commits to a feature branch per journey; merge to `staging` only after self-test.
- **17:30** — staging demo link and a 5-line status update to the client (done / next / need from client).
- **End of day** — update tracker: % complete per product, open bugs, risks.

9.2 Dependencies on the client (must arrive by Day 1–2)

Item	Needed for
Reference UI files / links, logo, brand colours, fonts	Re-skin (Section 8)
Final service list, categories, prices, commission and fee rules	Catalog and pricing
Razorpay account (test + live keys)	Payments
Google Maps API key with Places, Maps JS, Directions enabled and billing on	Maps & ETA
SMS gateway account, DLT sender ID and OTP template approval	Production OTP (DLT approval can take days — start immediately)
Firebase project for push	Notifications
Domain, hosting access, MongoDB Atlas (or server) access	Deployment
Legal text (T&C, privacy, refund), contact details	Website & app

10. Team, Responsibilities & Communication

10.1 Roles

Role	Responsibility	Strength
Project Lead (Aryan)	Scope, plan, client communication, daily reporting, risk & change control, final sign-off.	1
Frontend Developers	Re-skin, user/vendor/admin/website pages, tracking map UI, PWA.	2 (user+website, vendor+admin)
Backend Developer	Tracking service, security fixes, config, seed data, payments/notification wiring.	1
UI/UX Designer	Screen mapping, tokens, assets, design QA.	1 (part-time)
QA Engineer	Test cases, regression, device testing, UAT support.	1 (from Day 3)
DevOps (part-time)	Environments, CI, deployment, SSL, monitoring, backups.	1 (Day 2, 9, 10)

10.2 RACI

Activity	Lead	FE	BE	Design	QA	DevOps	Client
Scope freeze	R	C	C	C	I	I	A
UI re-skin	A	R	I	R	C	-	C
Live tracking	A	R	R	C	C	I	I
Payments & keys	A	C	R	-	C	C	R
Testing & UAT	A	C	C	I	R	I	R
Deployment	A	C	C	-	C	R	C
Go-live approval	R	I	I	I	C	I	A

R = Responsible, A = Accountable, C = Consulted, I = Informed.

10.3 Communication protocol

- **Single channel** for client queries (WhatsApp group / Slack); decisions confirmed in writing.
- **Blocker rule:** any blocker older than 2 hours is escalated to the Project Lead.
- **Client response SLA:** approvals and assets within 4 working hours to protect the timeline.
- **Demo rule:** every demo is on the staging URL, not on a developer machine.

11. Quality Assurance & UAT SOP

11.1 Test levels

Level	What is covered	Owner / When
Developer self-test	Feature works on desktop and phone viewport; no console errors.	Developer / each commit
Functional (QA)	Every feature in Section 6 against its acceptance rule.	QA / Day 3–8
Integration	Razorpay, SMS OTP, FCM push, Google Maps, Cloudinary uploads.	QA + BE / Day 4–7
Real-device	Android Chrome, iOS Safari, one low-end Android; GPS actually moving (walk/drive test).	QA / Day 6–8
Security	Auth on every route, role checks, socket validation, rate limits, no secrets in client bundle.	BE + Lead / Day 9
Performance	50 concurrent users smoke test; page load < 3 s on 4G; Lighthouse ≥ 80.	QA + DevOps / Day 8–9
UAT	Client executes the scripted journeys below on staging.	Client / Day 8

11.2 Priority-1 test scenarios

ID	Scenario	Expected result
T01	New user signs up with OTP	Account created, token issued, lands on home.
T02	Instant booking with online payment	Razorpay success → booking BROADCASTING; vendor receives popup < 3 s.
T03	Vendor accepts	User sees provider card; booking ACCEPTED; admin dashboard updates.
T04	Live tracking while EN_ROUTE	Marker moves; ETA updates; refresh restores position.
T05	Start & end OTP	Wrong OTP rejected; correct OTP advances status.
T06	Cash booking completion	Cash collected recorded; commission added to vendor dues.
T07	No provider accepts	After timeout booking FAILED; discount refunded; user notified.
T08	Cancellation by user / vendor	Correct refund/penalty per settings.
T09	Vendor KYC → admin approval	Status changes; vendor can go online only after approval.
T10	Withdrawal request → approve	Balance debited; status and remark visible to vendor.
T11	Role isolation	User token cannot call vendor/admin APIs (403).
T12	Location spoof attempt via socket	Rejected; no event delivered to other users.
T13	Rating after completion	Stored; average rating updates.
T14	Admin changes commission %	New bookings use new value; old bookings unchanged.
T15	Website CTAs	All links route correctly; forms submit; mobile layout intact.

11.3 Defect handling

Severity	Definition	Rule
S1 Critical	Money wrong, booking blocked, security hole, crash on core journey	Fix same day; blocks go-live
S2 Major	Feature broken with workaround	Fix before UAT close
S3 Minor	UI glitch, wording	Fix if time; else backlog

Exit criteria for UAT: 100% P1 scenarios pass, no open S1/S2, S3 list agreed with client.

12. Deployment, Go-live & Rollback SOP

12.1 Environments

Env	Purpose	Notes
Local	Developer work	Local MongoDB, test keys
Staging	QA, demos, UAT	Mirrors production; Razorpay test mode; seeded demo data
Production	Live customers	Separate DB, keys and domain; backups on

12.2 Pre-deployment checklist

- `NODE_ENV=production` ; **OTP bypass and dev OTP logging disabled** (`OTP_BYPASS_LAST6` , `OTP_DEV_LOG`).
- New strong `JWT_SECRET` ; admin seed password changed; default seed credentials removed.
- All secrets rotated and stored in the host's secret manager; `.env` and Firebase JSON not in the repository or hand-over zip.
- CORS restricted to the real domains; Helmet on; API rate-limit on auth and booking endpoints.
- MongoDB Atlas (or managed DB) with IP allow-list, automated daily backup, indexes verified (2dsphere on booking location, provider location).
- Razorpay live keys + webhook; Google Maps key restricted by HTTP referrer / IP; SMS DLT template approved.
- Utility scripts (`check_*` , `fix_*` , `test*`) excluded from the production build.
- Error logging (e.g. Sentry) and uptime monitoring configured.

12.3 Hosting plan

Component	Recommended hosting
Website + User/Vendor/Admin web apps	Vercel/Netlify static hosting (project already has `vercel.json`), custom domains with SSL
API + Socket.io	Node server on VPS / Render / Railway / AWS with **sticky WebSocket support** and process manager (PM2)
Database	MongoDB Atlas (M10 or higher for production)
Media	Cloudinary

12.4 Go-live runbook

1. Code freeze on the release branch; tag `v1.0.0` .
2. Take database snapshot (if migrating data) and record current DNS values.
3. Deploy API → run health check → run seed (categories, settings, admin) once.
4. Deploy web apps with production environment variables (`VITE_API_URL` , Maps, Razorpay key ID).
5. Switch DNS; verify SSL; verify WebSocket connects on the production domain.
6. Smoke test T01–T05 and T15 on production with a real ₹1–10 transaction and refund.
7. Announce to client; start hyper-care.

12.5 Rollback

- **Trigger:** any S1 in the first 2 hours, or payment/tracking failure rate > 5%.
- **Action:** redeploy previous tag (static apps are instant on Vercel; API via PM2 / previous image), restore DB snapshot only if data corruption is confirmed.
- **Owner:** DevOps executes, Project Lead decides, client informed immediately.

12.6 Hyper-care (first 7 days after go-live)

- Daily review of error logs, failed payments, stuck bookings (cron already auto-fails broadcasts after 10 min).
- Same-day response for S1/S2 issues; daily 5-line health report to client.
- Hand-over: admin user guide, vendor onboarding guide, API docs, credentials register (delivered securely), and one live training session.

13. Risks, Assumptions, Dependencies & Out-of-Scope

13.1 Assumptions

- The delivered system is built on the existing codebase; it is not a rewrite.
- "Application" means a responsive web app installable as a PWA. Native Android/iOS store apps are separate work.
- The reference UI is of mobile-app scale (about 15–25 screens). A much larger design needs re-estimation.
- The client supplies all third-party accounts and keys in Day 1–2.
- One city/zone and one language (English) at launch; Hindi/other languages are additive.
- The business domain of the client matches the "book a provider → track → pay" model of the existing platform. If the client's domain differs materially (e.g. car-specific inventory, listings, community feed), those parts are additional scope.

13.2 Risk register

#	Risk	Impact	Likelihood	Mitigation
1	Reference UI is larger/different from existing flows	High	Med	Screen mapping on Day 1; priority order; defer low-priority screens.

2	SMS DLT template approval delays OTP	High	Med	Apply immediately; keep staging bypass; escalate with gateway.
3	Browser geolocation stops when phone locks	High	High	Wake-lock + keep-screen-on prompt; communicate limitation; native wrapper as phase 2.
4	Google Maps billing / key not ready	High	Med	Client to enable billing Day 1; fall back to OSM/Leaflet for tracking map.
5	Scope creep from "features as discussed"	High	High	Written feature list signed on Day 1; change control (Section 14).
6	Money-flow defects (commission, refunds)	High	Low	Reconcile sample bookings on Day 4; S1 rule; real ₹1 test on production.
7	Secrets exposed in the repo	High	Med	Rotate all keys, purge from history, use secret manager.
8	Concurrency / socket scaling on single server	Med	Low	Single instance adequate for launch; Redis adapter if scaling out.
9	Client feedback arrives late	Med	Med	4-hour response SLA; auto-approval after 24 h silence for non-critical items.
10	Team member unavailable	Med	Low	Shared ownership per journey; buffer Day 9–10.

13.3 Out of scope for this engagement

- Native Android/iOS apps and store publishing (Play Store / App Store listing).
- In-app chat or voice calling beyond tap-to-call.
- Multi-language, multi-currency, multi-city expansion tooling.
- Automated accounting/ERP integration, GST e-invoicing.
- Marketing site blog/CMS beyond the pages listed; paid-ads setup.
- Corporate/B2B panel and BuildMart marketplace, unless confirmed in Day 1 scope freeze.
- Data migration from any legacy system.

13.4 Post-launch roadmap (suggested)

Phase	Items
Phase 2 (weeks 3–6)	Native apps (React Native / Capacitor wrapper), background GPS, in-app chat, surge/dynamic pricing, coupons.
Phase 3 (months 2–3)	Multi-city operations, advanced analytics, vendor ranking & auto-assignment, subscription marketing, WhatsApp notifications.

14. Change Control, Acceptance & Sign-off

14.1 Change-control procedure

1. **Raise:** any new or changed requirement is written in the change log (what, why, who asked).
2. **Assess:** Project Lead estimates effort, timeline and risk impact within 4 working hours.
3. **Decide:** client chooses one — (a) accept with extra days, (b) swap with an equal-sized item, (c) move to Phase 2.
4. **Record:** approved changes update this SOP version and the day plan.

> **Rule:** after Day 1 freeze, a request that adds more than 0.5 day of work is never absorbed silently — it is either swapped or moves the date.

14.2 Definition of Done (per feature)

- Implemented and merged to staging; no console or server errors.
- Works on mobile viewport and one real phone.
- Role/permission checks in place; inputs validated.
- Test scenario(s) pass; demo shown to Project Lead.
- Wording and visuals match the approved design.

14.3 Final acceptance criteria

#	Criterion	Verified by
1	User App, Vendor App, Admin Panel and Website are live on client domains with HTTPS.	Client
2	Complete journey (book → pay → live track → OTP complete → rate) works on production with a real transaction.	Client + QA

3	Vendor onboarding, approval, job handling and withdrawal verified.	Client + QA
4	Admin can configure fees, catalog, zones, content and see live bookings.	Client
5	UI matches the approved reference design on agreed priority screens.	Client
6	No open S1/S2 defects; S3 list accepted.	QA
7	Secrets rotated, backups on, monitoring active, rollback tested.	DevOps
8	Documentation and training delivered.	Lead

14.4 Open decisions to close on Day 1

#	Decision	Recommended default
1	Keep Corporate (B2B) panel and BuildMart marketplace?	Hide behind flags
2	Is the individual worker (`labour`) a separate app, or only Vendor?	Vendor only for launch
3	Pricing model: fixed per service or per hour/day? Commission % and platform fee?	Fixed per service + % commission
4	Payments: online only, or cash allowed?	Both, with cash-dues limit
5	Live tracking by web/PWA or native app?	PWA now, native in Phase 2
6	Launch city / zone and service radius	Single zone, 10 km
7	Reference UI: full screen list and priority	Provide on Day 1

14.5 Sign-off

Name / Role	Signature	Date
Client Representative		
Project Lead — Aryan Pathak		
Management		

This SOP is derived from a static review of the repository and the client conversation. Items marked “to be confirmed” or listed in Section 14.4 must be validated with the client before scope freeze.